R3.04 Qualité de développement :

Le *design pattern Data Access Object*

Présentation (1)

- Problème : dans une application à objets, qui interagit avec une solution de persistance, où mettre le code permettant d'accéder aux données ?
- Solution de persistance : une façon de stocker des données (bdd, xml, mémoire, fichiers, etc.)

Présentation (2)

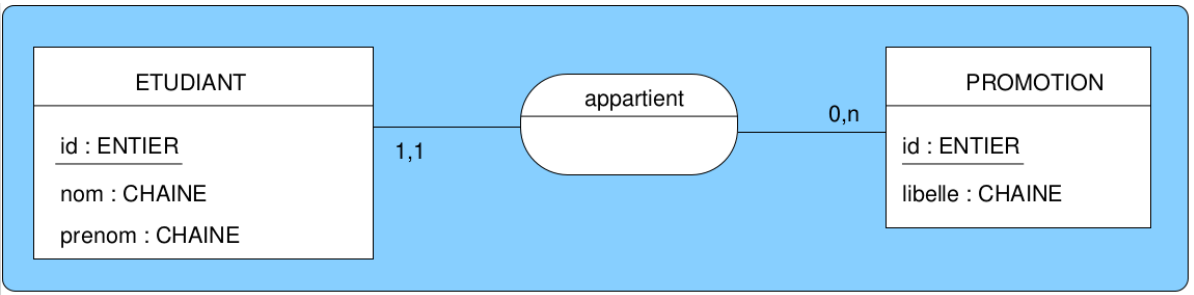
Questions qui se posent :

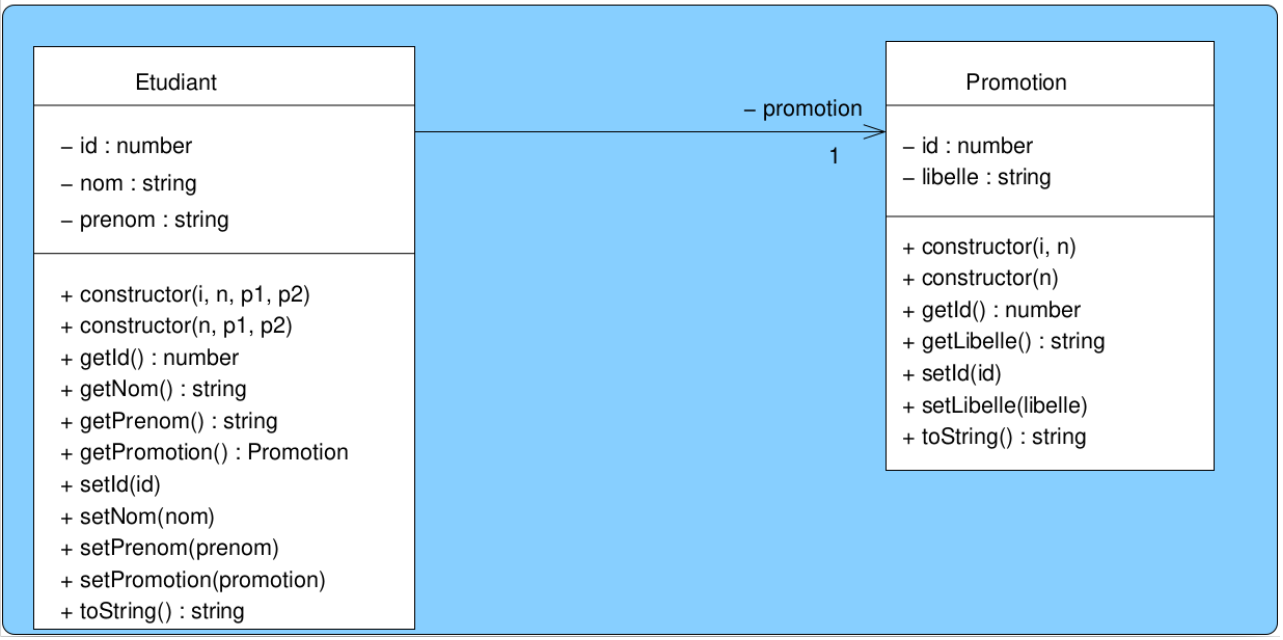
- Comment obtenir une application la plus modulaire possible, la plus facilement modifiable et la plus réutilisable ?
- Que faire pour simplifier un futur changement de base de données cible ? (MySQL → Oracle)
- Que faire pour simplifier un futur changement de solution de persistance ? (MySQL → XML ; MySQL → NoSQL ; etc.)

Solution : DAO

DAO niveau 1 (1)

- A utiliser si on sait que la solution de persistance ne changera jamais...
- Trois éléments :
  - Une classe de connexion
  - Pour chaque table :
    - 1 classe métier
    - 1 classe DAO





DAO niveau 1 (2)

- Du côté BDD : schéma E/A
- Application : objets métier correspondants

DAO niveau 1 (3) - Classes métier

```
export class Promotion {

    private _id : number;
    private _libelle : string;

    public constructor(libelle : string, id = -1) {

        this.id = id;
        this.libelle = libelle;
    }

    public get id(): number {
        return this._id;
    }

    public get libelle(): string {
        return this._libelle;
    }
```

DAO niveau 1 (4) - Classes métier

```
    public set id(id: number) {
        this._id = id;
    }

    public set libelle(libelle: string) {

        if (libelle.trim().length < 2) {
            throw new Error("Le libellé de promotion doit avoir au moins deux lettres !");
        }
        this._libelle = libelle.trim();
    }

    public toString() : string {

        return this._libelle;
    }

}
```

DAO niveau 1 (5) - Classes métier

```
export class Etudiant {

    private _id : number;
    private _nom : string;
    private _prenom : string;
    private _promotion : Promotion;

    public constructor(nom : string, prenom : string, promotion : Promotion, id = -1) {

        this.id = id;
        this.nom = nom;
        this.prenom = prenom;
        this.promotion = promotion;
    }

    public get id(): number {
        return this._id;
    }

    public get promotion(): Promotion {
        return this._promotion;
    }

    ...

}
```

DAO niveau 1 (6) - Classes métier

- Servent à créer les objets permettant de stocker en mémoire les n-uplets des tables
- On parle d'ORM (*Object-relational mapping*) à chaque table est associé un objet et inversement
- Les méthodes qui vont réaliser des `Insert`, `Update` ou `Delete` prendront un objet métier en paramètre
- Méthodes de `Select` : objets métier en retour de fonction

DAO niveau 1 (7) - Classes métier VS DAO (1)

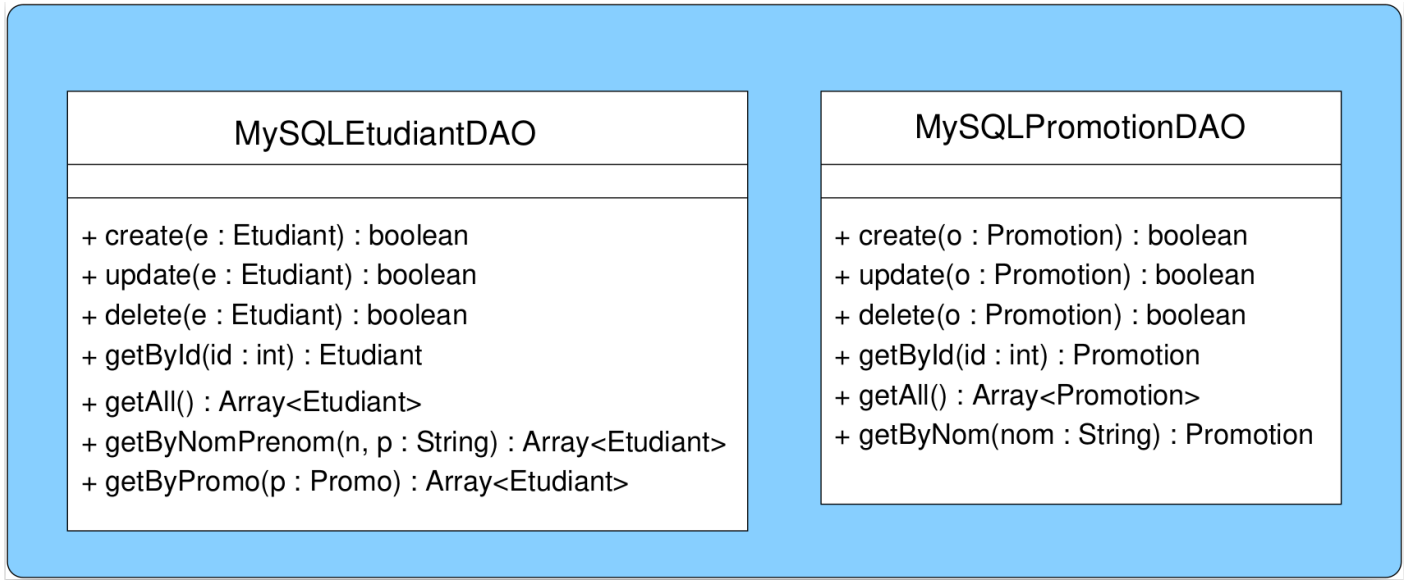
- Où mettre le SQL ?
- Dans les classes métier ?
- Gros inconvénients : en cas de changement de solution de persistance (SGBD → XML)
  - le code de persistance (`select`, `update`, etc.) change complètement
  - l'objet métier, lui, reste le même
- Rappel de bonne pratique **principe ouvert/fermé** : l'ajout de code est plus sûr que la modification

DAO niveau 1 (8) - Classes métier VS DAO (2)

- Conséquence : le code de persistance est isolé dans une classe DAO par table, pour le séparer du code métier
- Chaque classe DAO contient les méthodes CRUD
  - Create (insert into)
  - Request (select from)
  - Update
  - Delete

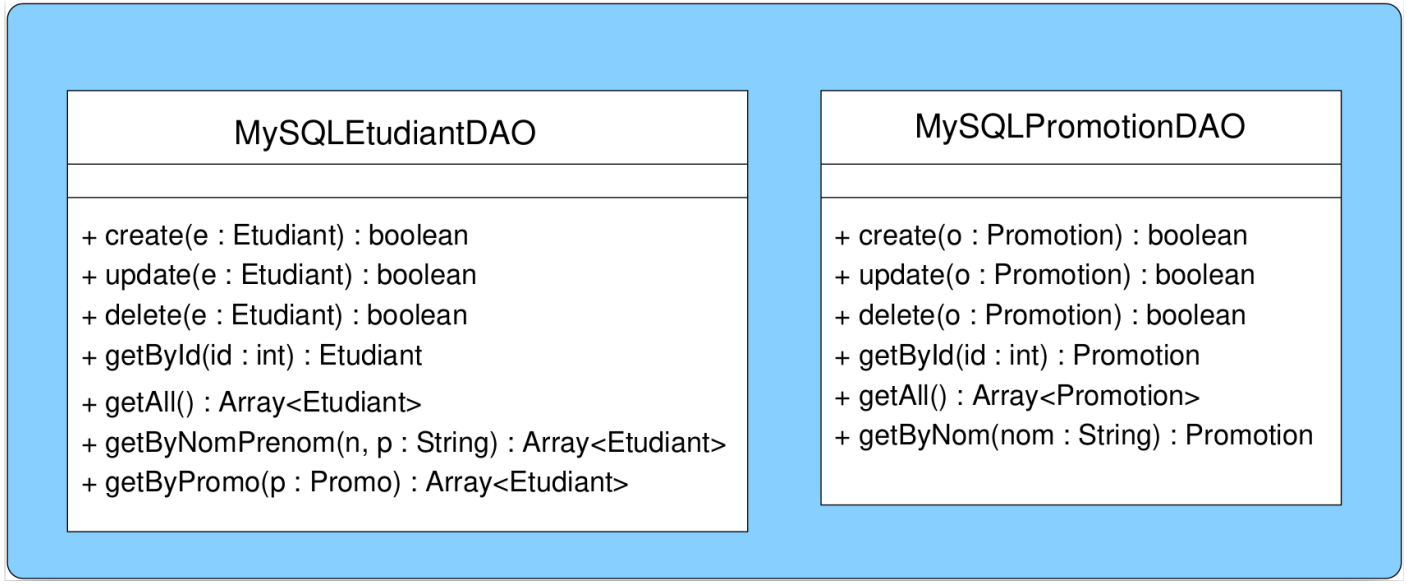
DAO niveau 1 (9) - Classes DAO (1)

- Autant de Request que nécessaire : en fonction des fonctionnalités requises
  - `getAll`, `getById` indispensables
  - Autres `getBy…` selon les besoins
- Une méthode de chaque pour Create, Update, Delete



DAO niveau 1 (10) - Classes DAO (2)

- Méthode delete : une seule façon de supprimer un objet/n-uplet
- Méthode insert : idem
- Méthode update : tous les attributs de l'objet sont mis à jour dans la base
- Toutes ces méthodes renvoient un booléen indiquant si l'opération s'est bien passée



DAO niveau 1 (11) - Généricité (1)

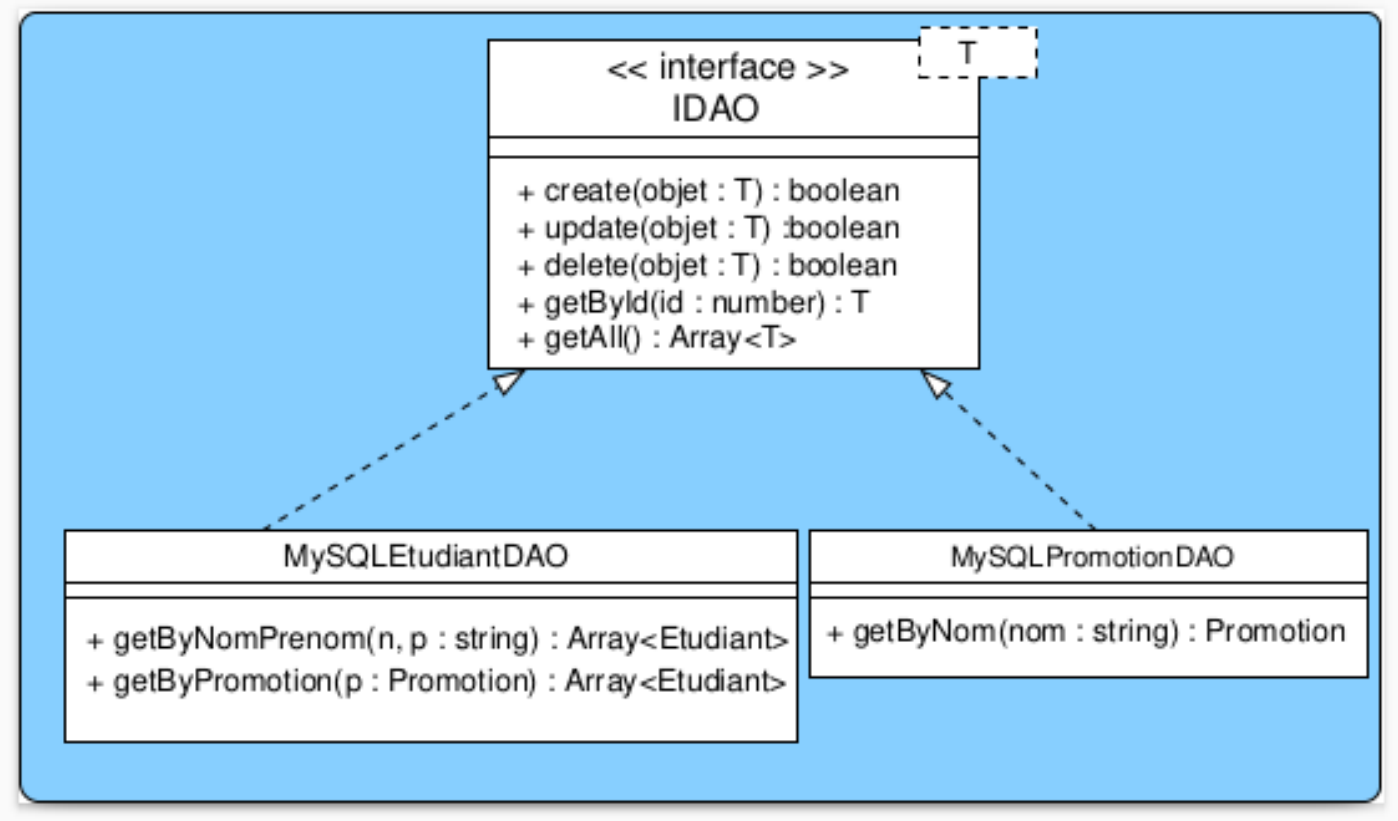
- Une interface pour spécifier les méthodes communes

```
export interface IDAO<T> {

    getAll() : Promise<T[]>;
    getById(id : number) : Promise<T | null>;
    create(objet : T) : Promise<boolean>;
    update(objet : T) : Promise<boolean>;
    delete(objet : T) : Promise<boolean>;
}
```

DAO niveau 1 (12) - Généricité (2)

Diagramme de classe incluant l'interface générique



DAO niveau 1 (13) - Généricité (3)

- Est-ce une bonne idée ?
- Tous les objets DAO implémentent l'interface `IDAO`, donc tous les objets DAO doivent implémenter les cinq méthodes
- Or : dans une application, on a des données en lecture/écriture, mais aussi beaucoup d'objets en lecture seule
- Exemple : appli de gestion des absences
  - Les étudiants, promotion (1a, 2a, etc.) sont gérés par une autre appli, on a uniquement des méthode `Request` dans l'appli

DAO niveau 1 (14) - Généricité (4)

- Alors que la table des absences, celle des justificatifs, ... ont besoin de méthodes CRUD
- Dans ce cas, il vaut mieux éviter la généricité, ne pas implémenter `IDAO` **sur toutes les classes DAO**
- Sinon on doit implémenter des méthodes vides !

```
// CE QU'ON NE VEUT/DOIT PAS FAIRE si les promotions sont gérées par une autre application
export class MySQLPromotionDAO implements IDAO<Promotion>{
    ...

    // insertion obligatoire de code inutile et peu compréhensible...
    async update(objet : Promotion) : Promise<boolean> {

        return true;
    }

}
```

mysql / connexion au serveur

```
import { Client } from "https://deno.land/x/mysql@v2.12.1/mod.ts";

export class Connexion {
  public static async getConnection(): Promise<Client> {
    const c = await new Client().connect({
      hostname: "devbdd.iutmetz.univ-lorraine.fr",
      username: "laroche5_appli",
      password: "******",
      db: "laroche5_r304",
    });

    return c;
  }
}
```

mysql / requête SELECT

- `query` renvoie un tableau d'objets JSON

```
export class MySQLPromotionDAO implements IDAO<Promotion> {

    async getAll(): Promise<Promotion[]> {
        const liste: Promotion[] = [];

        const laConnexion = await Connexion.getConnection();
        const lignes = await laConnexion.query(
            "SELECT * FROM promotion ORDER BY id",
        );

        for (const ligne of lignes) {
            const p = new Promotion(ligne.libelle, ligne.id);
            liste.push(p);
        }
        laConnexion.close();

        return liste;
    }
```

mysql / requête C, U, D

- `query` renvoie un objet JSON avec deux attributs `affectedRows` et `lastInsertId`

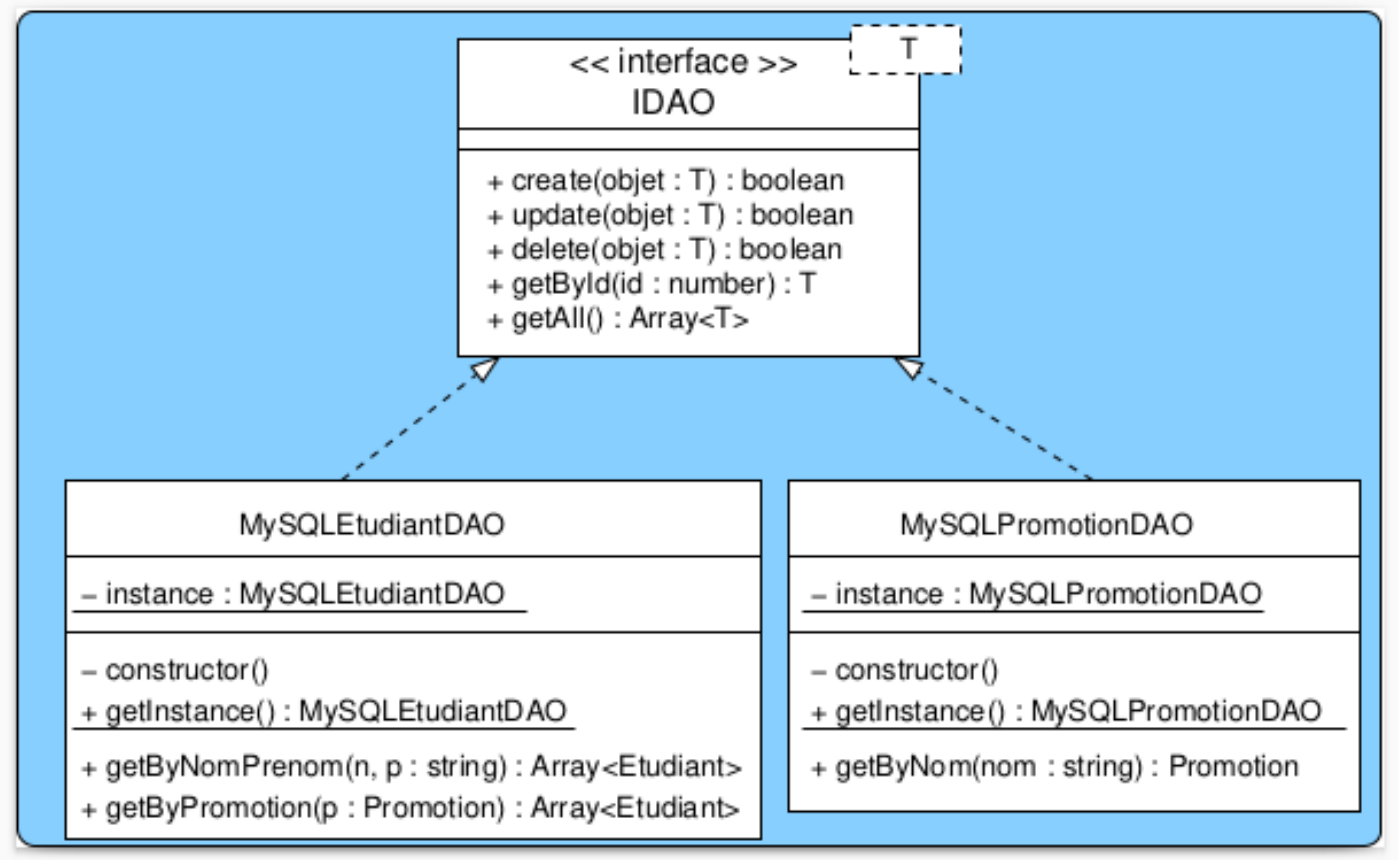
```
async create(objet: Promotion): Promise<boolean> {
    let ok = true;
    const laConnexion = await Connexion.getConnection();
    const resultat = await laConnexion.query(
        "INSERT INTO promotion (libelle) values (?)",
        [objet.libelle],
    );

    if (resultat.affectedRows === 1) {
        objet.id = resultat.lastInsertId;
    } else {
        ok = false;
    }
    laConnexion.close();

    return ok;
}
```

Classes DAO : singletons !

- Eviter de créer un objet `MySQLPromotionDAO` à chaque requête SQL
- Chaque classe DAO est un singleton ! (cf cours précédent)



Conclusion

- Nous avons répondu à la question : Où mettre le code de persistance ?
- Utilisation d'une solution d'ORM
- La classe générique `IDAO<T>` est à utiliser si nécessaire…
- Question sans réponse : comment préparer un futur changement de solution de persistance (on passe de MySQL à XML) ...